

Université Sultan Moulay Slimane

Faculté des Sciences et Techniques de Beni Mellal

Master : Artificial Intelligence & Digital Computation

Année universitaire : 2026–2027

COMPTE RENDU 3

TP3 : Prompt Engineering et LLM

Tokenisation, LangChain et Ollama, analyse de sentiment par prompting

---

Module : Système Distribué Agentique

Encadré par : Pr. Lhoussaine Daoudi

Fait par : Wissal Dahibi

Table des matières

1 Introduction et objectifs **3**

2 Tokenisation avec `tiktoken` **3**

2.1 Pourquoi s'intéresser aux tokens ? 3

2.2 Mise en œuvre 3

2.3 Observations 5

3 Utilisation d'un LLM avec LangChain **5**

3.1 Première tentative avec OpenAI 6

3.2 Alternative avec Ollama 6

4 Résultat obtenu **7**

5 Analyse de sentiment sur les avis IMDb **7**

5.1 Métriques d'évaluation 8

5.2 Chargement et préparation des données 8

5.3 Construction des prompts 10

5.3.1 Prompt Zero-shot 10

5.3.2 Prompt Few-shot 10

5.3.3 Prompt Chain-of-Thought 11

5.4 Évaluation des prompts avec Ollama 12

5.5 Analyse de la matrice de confusion 13

5.6 Répétition des évaluations et variabilité 13

6 Conclusion **15**

1 Introduction et objectifs

Ce compte rendu présente le travail réalisé lors du TP3, consacré au *prompt engineering*. Le TP s'intéresse aux tâches de classification, en particulier à l'analyse de sentiment : l'idée est de voir comment un LLM peut servir d'outil de classification et comment la façon de rédiger le prompt influence les résultats.

Le TP se déroule en deux temps. On commence par les bases : étude du découpage d'un prompt en tokens, puis appel d'un modèle de langage avec LangChain. On applique ensuite ces bases à une tâche de classification, l'analyse de sentiment, en comparant trois façons de rédiger le prompt. Pour l'environnement de travail : VS Code, uv, environnement virtuel.

2 Tokenisation avec `tiktoken`

2.1 Pourquoi s'intéresser aux tokens ?

Un LLM ne lit pas le texte tel quel : il travaille sur des *tokens*, c'est-à-dire des morceaux de mots, de ponctuation ou d'espaces, convertis en nombres entiers.

2.2 Mise en œuvre

On a installé la bibliothèque dans l'environnement existant avec `uv add tiktoken`, puis on a demandé quel encodage est associé au modèle `gpt-4o`. La réponse est `o200k_base`, qu'on a ensuite chargé directement.

```
import tiktoken

tokenizer = tiktoken.encoding_for_model("gpt-4o")
print(tokenizer.name)
tokenizer = tiktoken.get_encoding("o200k_base")
```

```
import tiktoken
```
[1] ✓ 0.0s

```
tokenizer=tiktoken.encoding_for_model("gpt-4o")
```
[2] ✓ 0.9s

```
print(tokenizer.name)
```
[3] ✓ 0.0s

```
o200k_base
```

FIGURE 1 – Chargement de tiktoken et encodage associé à gpt-4o

On encode ensuite un prompt, puis on affiche les tokens obtenus et leur nombre :

```
prompt = "Vous êtes un expert en marketing digital. C'est quoi un digital
    marketer ? "
tokens = tokenizer.encode(prompt)
print(tokens)
print(len(tokens))
```

```
[29038, 34077, 537, 8333, 469, 6686, 7058, 13, 363, 6616, 33399, 537,
 7058, 95641, 1423, 220]
16
```

```
prompt="Vous êtes un expert en marketing digital. C'est quoi un digital marketer ? "
```
] ✓ 0.0s

```
tokens=tokenizer.encode(prompt)
```
] ✓ 0.2s

```
print(tokens)
```
] ✓ 0.0s

```
[29038, 34077, 537, 8333, 469, 6686, 7058, 13, 363, 6616, 33399, 537, 7058, 95641, 1423, 220]
```

FIGURE 2 – Encodage du prompt et liste des tokens obtenus

Pour voir à quoi correspondent ces nombres, on décode chaque token séparément :

```
for token in tokens:
    t = tokenizer.decode_single_token_bytes(token).decode("utf-8")
    print(t, end='|')
```

```
Vous| êtes| un| expert| en| marketing| digital|.| C|'est| quoi| un| digital|
    marketer| ?| |
```

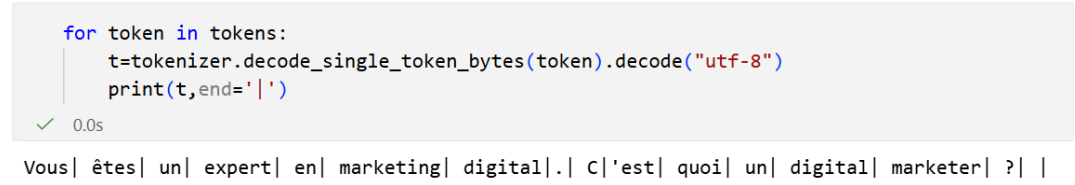


FIGURE 3 – Décodage des tokens un par un (séparateur |)

2.3 Observations

Le prompt contient 12 mots mais donne 16 tokens, ce qui montre que le découpage ne suit pas simplement les mots. Plusieurs choses ressortent de la sortie :

- la ponctuation (le point et le point d'interrogation) forme des tokens à part ;
- « C'est » est coupé en deux tokens, « C » et « 'est » ;
- les espaces sont rattachés au début du mot qui suit (« êtes », « expert ») et l'espace laissé à la fin du prompt compte aussi comme un token ;
- des mots courants comme « marketing » ou « digital » restent en un seul token.

3 Utilisation d'un LLM avec LangChain

LangChain fournit une interface commune pour appeler différents modèles de langage depuis Python. On a ajouté les bibliothèques nécessaires à l'environnement :

TABLEAU 1 – Bibliothèques installées pour ce TP

Bibliothèque	Installation	Rôle
tiktoken	`uv add tiktoken`	tokenisation des prompts
LangChain	`uv add langchain`	interface commune pour appeler les LLM
langchain-openai	`uv add langchain-openai`	connexion aux modèles OpenAI
langchain-ollama	`uv add langchain-ollama`	connexion aux modèles via Ollama

3.1 Première tentative avec OpenAI

On a utilisé `gpt-4o` via l'API d'OpenAI. La clé API, générée sur la plateforme OpenAI est stockée dans un fichier `.env` et chargée avec `load_dotenv`, ce qui évite de l'écrire dans le notebook. On fixe `temperature=0` pour obtenir des réponses les plus stables possible. L'appel se fait avec `invoke` en envoyant un message système et un message utilisateur .

```
from langchain_openai import ChatOpenAI
from dotenv.ipython import load_dotenv

load_dotenv(override=True)
llm = ChatOpenAI(model="gpt-4o", temperature=0)

resp = llm.invoke(input=[
    {"role": "system", "content": "Vous êtes un expert en marketing digital"},
    {"role": "user", "content": "C'est quoi un digital marketer ?"}
])
```

L'API d'OpenAI étant payante, on n'a pas pu obtenir de réponse du modèle. Il a donc fallu chercher une alternative.

3.2 Alternative avec Ollama

On a alors utilisé **Ollama**, qui permet d'accéder à des modèles de langage sans passer par l'API d'OpenAI. Après l'avoir installé, on a lancé un modèle cloud depuis PowerShell avec la commande `ollama run`, puis on a installé l'intégration LangChain avec `uv add langchain-ollama`.

Pour le code, on a choisi le modèle `gemma4:31b-cloud`. Le suffixe `-cloud` indique que le modèle est exécuté à distance et non sur l'ordinateur, ce qui permet d'utiliser un modèle de 31 milliards de paramètres sans GPU puissant. Le code est quasiment identique à celui d'OpenAI : seule la classe change.

```
from langchain_ollama import ChatOllama
from IPython.display import Markdown

llm2 = ChatOllama(model="gemma4:31b-cloud", temperature=0)

resp2 = llm2.invoke(input=[
    {"role": "system", "content": "Vous êtes un expert en marketing digital"},
    {"role": "user", "content": "C'est quoi un digital marketer ?"}
])

display(Markdown(resp2.content))
```

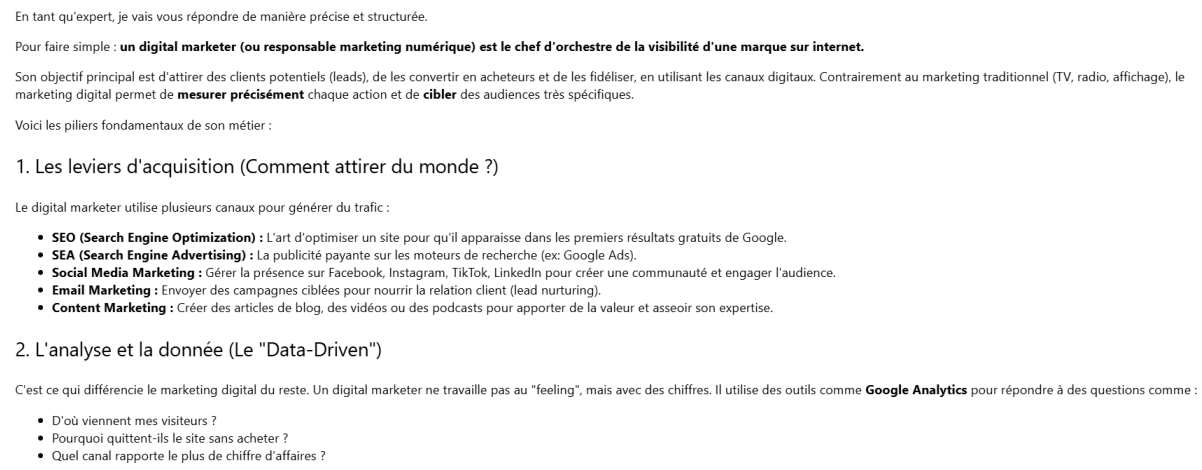
La réponse du modèle étant écrite en Markdown (titres, listes, gras), on l'affiche avec `display(Markdown(...))` plutôt qu'avec un simple `print`, ce qui donne un rendu lisible directement dans le notebook.

4 Résultat obtenu

Le modèle a produit une réponse structurée, qui commence par résumer le rôle du digital marketer comme celui qui gère la visibilité d'une marque sur internet. Il détaille ensuite plusieurs points :

- les leviers d'acquisition : SEO, SEA, réseaux sociaux, email marketing et marketing de contenu ;
- l'analyse de données, avec des outils comme Google Analytics ;
- l'optimisation de la conversion (CRO) et de l'expérience utilisateur ;
- les différents profils, entre généraliste (*T-shaped*) et spécialiste.

On remarque que le modèle adopte bien le rôle donné dans le message système : il répond en « expert » avec un ton assuré et une présentation organisée. Ce qui montre l'influence du prompt sur le style et le contenu de la réponse.



En tant qu'expert, je vais vous répondre de manière précise et structurée.

Pour faire simple : **un digital marketer (ou responsable marketing numérique) est le chef d'orchestre de la visibilité d'une marque sur internet.**

Son objectif principal est d'attirer des clients potentiels (leads), de les convertir en acheteurs et de les fidéliser, en utilisant les canaux digitaux. Contrairement au marketing traditionnel (TV, radio, affichage), le marketing digital permet de **mesurer précisément** chaque action et de **cibler** des audiences très spécifiques.

Voici les piliers fondamentaux de son métier :

1. Les leviers d'acquisition (Comment attirer du monde ?)

Le digital marketer utilise plusieurs canaux pour générer du trafic :

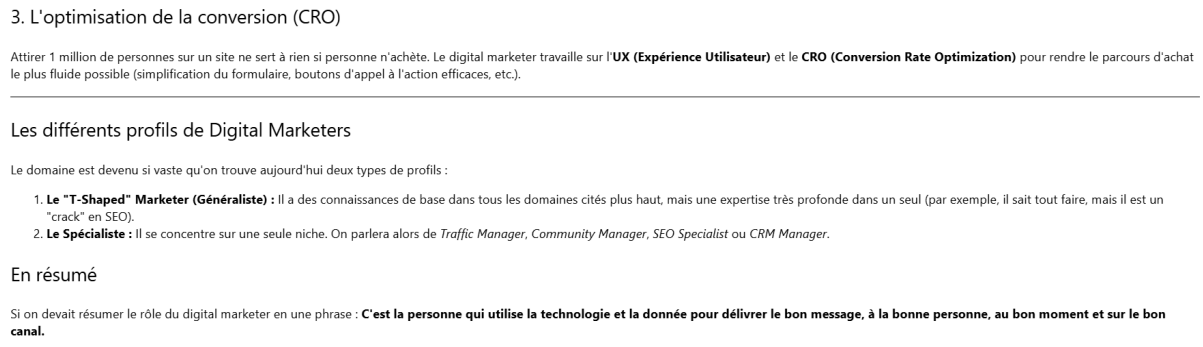
- **SEO (Search Engine Optimization) :** L'art d'optimiser un site pour qu'il apparaisse dans les premiers résultats gratuits de Google.
- **SEA (Search Engine Advertising) :** La publicité payante sur les moteurs de recherche (ex: Google Ads).
- **Social Media Marketing :** Gérer la présence sur Facebook, Instagram, TikTok, LinkedIn pour créer une communauté et engager l'audience.
- **Email Marketing :** Envoyer des campagnes ciblées pour nourrir la relation client (lead nurturing).
- **Content Marketing :** Créer des articles de blog, des vidéos ou des podcasts pour apporter de la valeur et asseoir son expertise.

2. L'analyse et la donnée (Le "Data-Driven")

C'est ce qui différencie le marketing digital du reste. Un digital marketer ne travaille pas au "feeling", mais avec des chiffres. Il utilise des outils comme **Google Analytics** pour répondre à des questions comme :

- D'où viennent mes visiteurs ?
- Pourquoi quittent-ils le site sans acheter ?
- Quel canal rapporte le plus de chiffre d'affaires ?

FIGURE 4 – Réponse de `gemma4:31b-cloud` affichée en Markdown dans le notebook



3. L'optimisation de la conversion (CRO)

Attirer 1 million de personnes sur un site ne sert à rien si personne n'achète. Le digital marketer travaille sur l'**UX (Expérience Utilisateur)** et le **CRO (Conversion Rate Optimization)** pour rendre le parcours d'achat le plus fluide possible (simplification du formulaire, boutons d'appel à l'action efficaces, etc.).

Les différents profils de Digital Marketers

Le domaine est devenu si vaste qu'on trouve aujourd'hui deux types de profils :

1. **Le "T-Shaped" Marketer (Généraliste) :** Il a des connaissances de base dans tous les domaines cités plus haut, mais une expertise très profonde dans un seul (par exemple, il sait tout faire, mais il est un "crack" en SEO).
2. **Le Spécialiste :** Il se concentre sur une seule niche. On parlera alors de *Traffic Manager*, *Community Manager*, *SEO Specialist* ou *CRM Manager*.

En résumé

Si on devait résumer le rôle du digital marketer en une phrase : **C'est la personne qui utilise la technologie et la donnée pour délivrer le bon message, à la bonne personne, au bon moment et sur le bon canal.**

FIGURE 5 – SUITE : Réponse de `gemma4:31b-cloud` affichée en Markdown dans le notebook

5 Analyse de sentiment sur les avis IMDb

Après avoir exploré la tokenisation et testé l'appel à un modèle de langage, on passe à une tâche de classification : l'analyse de sentiment. Il s'agit d'attribuer à chaque avis de film

l'étiquette `positive` ou `negative` à partir de son texte. Contrairement à un apprentissage classique, on n'entraîne aucun modèle ici mais on étudie la capacité d'un LLM à réaliser cette classification en suivant différentes consignes, c'est-à-dire différents prompts.

Le jeu de données utilisé est le fichier `IMDBDataset.csv`, qui contient 50 000 avis de films dans la colonne `review` et leur sentiment dans la colonne `sentiment`.

5.1 Métriques d'évaluation

Chaque avis reçoit une seule étiquette parmi deux, on peut donc comparer directement la prédiction du modèle à l'étiquette de référence. Deux métriques sont utilisées :

- l'**accuracy**, c'est-à-dire la proportion de prédictions correctes ;
- le **score micro-F1**, qui cumule les vrais positifs (TP), les faux positifs (FP) et les faux négatifs (FN) sur l'ensemble des classes, puis calcule la précision P et le rappel R globaux .

Dans le notebook, ces scores sont calculés avec les fonctions `f1_score` et `accuracy_score` de scikit-learn. Comme chaque avis ne reçoit qu'une seule étiquette le micro-F1 et l'accuracy donnent la même valeur ici.

5.2 Chargement et préparation des données

On importe d'abord les bibliothèques nécessaires, puis on charge le fichier avec `pandas`. On garde les deux colonnes utiles, on supprime les lignes incomplètes et on normalise les étiquettes pour avoir une représentation uniforme (`positive` et `negative`). Le dataset contient bien 50 000 avis, répartis de façon équilibrée : 25 000 positifs et 25 000 négatifs.

```
df = pd.read_csv(CSV_PATH, sep=None, engine="python")
df.columns = [str(col).strip().lower() for col in df.columns]
df = df[["review", "sentiment"]].dropna().copy()
df["review"] = df["review"].astype(str).str.strip()

# Normalisation des étiquettes (extrait du dictionnaire utilisé)
label_mapping = {"positive": "positive", "pos": "positive", "1": "positive",
                 "negative": "negative", "neg": "negative", "0": "negative"}
df["sentiment"] = (df["sentiment"].astype(str).str.strip().str.lower()
                   .map(label_mapping))
```

```
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

FIGURE 6 – Répartition des sentiments dans le dataset IMDb

On sépare ensuite les données en deux ensembles avec un découpage 80/20 stratifié afin de conserver les proportions des deux classes. Les 40 000 avis du premier ensemble servent de réservoir pour construire les exemples des prompts et les 10 000 avis du second servent de réservoir pour l'évaluation.

```
train_df, test_df = train_test_split(
    df, test_size=0.20, random_state=42, stratify=df["sentiment"]
)
```

Pour limiter le nombre d'appels au modèle cloud et garder un temps d'exécution raisonnable, on ne conserve que dix avis de référence : cinq positifs et cinq négatifs tirés dans l'ensemble de test avec `random_state=42`, pour que le tirage soit identique à chaque exécution.

```
gold_df = test_df.groupby("sentiment", group_keys=False).sample(
    n=5, random_state=42
)
```

```
Exemples disponibles : 40000
Avis de référence : 10
```

	review	sentiment
46566	In its way, Mister Foe (originally, and more a...	negative
36683	OK, the story - a simpleminded loony enters a ...	negative
21163	I was honestly surprised by Alone in the Dark....	negative
14920	I totally disagreed with those comments which ...	negative
5683	I wish that all the mockumentaries and horror ...	negative

FIGURE 7 – Séparation des données et sélection des dix avis de référence

5.3 Construction des prompts

On prépare trois prompts pour observer l'effet de consignes différentes sur la classification. Dans les trois cas l'avis à classer est envoyé dans un message utilisateur construit à partir d'un même modèle .

```
user_message_template = """
Classify the sentiment of the following movie review.

Movie review:
```{movie_review}```
"""
```

5.3.1 Prompt Zero-shot

Le premier prompt demande directement au modèle de classer un avis, sans lui donner d'exemple. Le message système impose une réponse limitée à l'étiquette.

```
zero_shot_system_message = """
Classify the sentiment of movie reviews as 'positive' or 'negative'.
The review will be delimited by triple backticks.
Answer only 'positive' or 'negative'.
Do not explain your answer.
"""
zero_shot_prompt = [{"role": "system", "content": zero_shot_system_message}]
```

5.3.2 Prompt Few-shot

Le message système reste le même, mais on lui ajoute des exemples d'avis déjà classés. La fonction `create_examples` tire quatre avis positifs et quatre avis négatifs, puis les mé-
```

lange. Équilibrer les classes et mélanger l'ordre permet d'éviter deux biais connus des prompts few-shot : le biais de l'étiquette majoritaire et le biais de récence, c'est-à-dire l'influence des exemples placés en fin de prompt. La fonction `create_prompt` transforme ensuite chaque exemple en deux messages (l'avis côté utilisateur, l'étiquette côté assistant). On obtient un prompt de 17 messages : le message système et 8 exemples de 2 messages chacun.

```
def create_examples(dataset, n=4, random_state=None):
    positives = dataset[dataset["sentiment"] == "positive"]
    negatives = dataset[dataset["sentiment"] == "negative"]
    examples = pd.concat([
        positives[["text", "sentiment"]].sample(n=n, random_state=random_state
    ),
        negatives[["text", "sentiment"]].sample(n=n, random_state=random_state
    )
    ])
    # mélange aléatoire des exemples
    shuffled = examples.sample(frac=1, random_state=random_state)
    return shuffled.to_json(orient="records", force_ascii=False)

def create_prompt(system_message, examples, user_message_template):
    prompt = [{"role": "system", "content": system_message}]
    for ex in json.loads(examples):
        prompt.append({"role": "user",
                       "content": user_message_template.format(
                           movie_review=ex["text"])})
        prompt.append({"role": "assistant", "content": ex["sentiment"]})
    return prompt
```

5.3.3 Prompt Chain-of-Thought

Le troisième prompt reprend les mêmes exemples que le few-shot, mais son message système change : il demande d'examiner attentivement l'avis et de considérer son sentiment général avant de choisir l'étiquette.

```
cot_system_message = """
Classify the sentiment of movie reviews as 'positive' or 'negative'.
The review will be delimited by triple backticks.
Carefully examine the review and consider its overall sentiment
before choosing the appropriate label.
The final answer must contain only: 'positive' or 'negative'.
Do not provide an explanation in the final answer.
"""
cot_few_shot_prompt = create_prompt(
    cot_system_message, examples, user_message_template
)
```

Comme la réponse finale doit rester limitée à l'étiquette, le raisonnement n'apparaît pas dans la sortie du modèle : c'est donc une version légère du Chain-of-Thought. Ces trois prompts permettent de comparer une classification sans exemples, une classification guidée par des exemples, et une classification qui ajoute une consigne d'analyse.

```
Nombre de messages du prompt : 17
```

FIGURE 8 – Taille du prompt Few-shot (17 messages)

5.4 Évaluation des prompts avec Ollama

On évalue les trois prompts avec le modèle `gemma4:31b-cloud` via `ChatOllama` . Les prompts sont ainsi comparés avec le même modèle.

La fonction `evaluate_prompt` parcourt les dix avis de référence. Pour chacun, elle ajoute l'avis au prompt, envoie le tout au modèle avec `invoke`, puis extrait l'étiquette de la réponse avec la fonction `extract_sentiment`. Cette dernière regarde d'abord si la dernière ligne de la réponse est exactement une étiquette. Sinon, elle prend la dernière étiquette mentionnée, et renvoie `unknown` si elle n'en trouve aucune. Les prédictions sont enfin comparées aux étiquettes réelles pour calculer le micro-F1, l'accuracy et le nombre de prédictions `unknown`.

```
response = model.invoke(prompt + user_input)
prediction = extract_sentiment(response.content)

micro_f1 = f1_score(ground_truths, predictions, average="micro")
accuracy = accuracy_score(ground_truths, predictions)
```

La fonction est appliquée successivement aux prompts Zero-shot, Few-shot et CoT. Les résultats sont regroupés dans le tableau suivant.

TABLEAU 2 – Résultats de l'évaluation des trois prompts sur les dix avis de référence

Prompt	Micro-F1	Accuracy	Prédictions inconnues
Zero-shot	1,000	1,000	0
Few-shot	1,000	1,000	0
CoT Few-shot	1,000	1,000	0

Sur ces dix avis, les trois prompts obtiennent un score de 1,0 et aucune réponse n'a été classée `unknown` : toutes les réponses du modèle étaient donc exploitables automatiquement, et toutes les prédictions correspondaient aux étiquettes de référence.

```
Évaluation du prompt : Zero-shot
Évaluation du prompt : Few-shot
Évaluation du prompt : CoT Few-shot
```

prompt	micro_f1	accuracy	unknown_predictions
Zero-shot	1.0	1.0	0
Few-shot	1.0	1.0	0
CoT Few-shot	1.0	1.0	0

FIGURE 9 – Comparaison des scores micro-F1 et accuracy des trois prompts

5.5 Analyse de la matrice de confusion

On trace la matrice de confusion du prompt ayant obtenu le meilleur micro-F1. Les trois prompts étant à égalité à 1,0, c'est le premier de la liste qui est retenu, soit le Zero-shot. La matrice compare les étiquettes réelles aux étiquettes prédites, avec une colonne supplémentaire `unknown` pour les réponses non reconnues. Ici, les cinq avis négatifs et les cinq avis positifs sont tous bien classés et rien n'est prédit `unknown`.

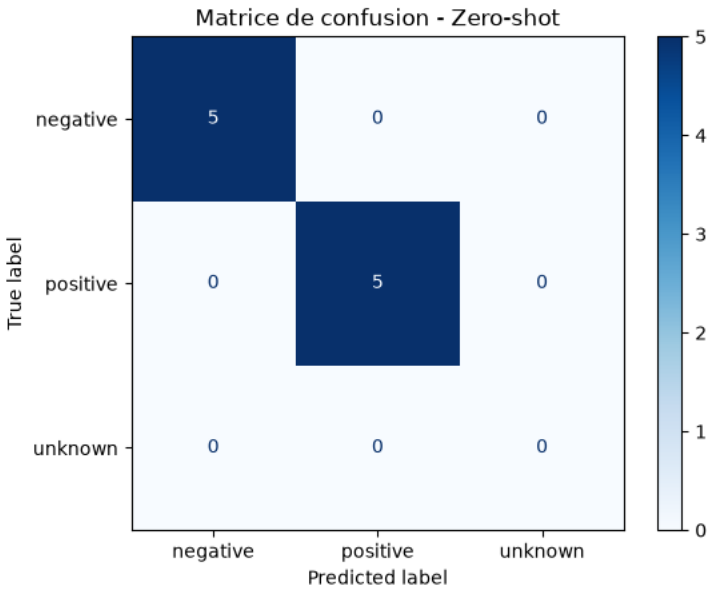


FIGURE 10 – Matrice de confusion du prompt Zero-shot

5.6 Répétition des évaluations et variabilité

Les prompts Few-shot et CoT dépendent des exemples tirés pour les construire ; on s'attend donc à une certaine variabilité selon ce tirage. Pour la mesurer ; On répète l'évaluation en changeant à chaque fois les exemples (`random_state=100 + run`). À chaque répétition, les deux

prompts sont reconstruits avec les mêmes exemples de sorte que seul le message système les distingue, puis évalués sur les mêmes dix avis de référence. Le support propose 10 répétitions ; on se limite ici à 3 pour réduire le nombre d'appels au modèle cloud (3 répétitions, 2 prompts et 10 avis).

```
num_eval_runs = 3
few_shot_scores, cot_scores = [], []

for run in range(num_eval_runs):
    examples_run = create_examples(examples_df, n=n_shots,
                                   random_state=100 + run)
    few_prompt = create_prompt(few_shot_system_message,
                               examples_run, user_message_template)
    cot_prompt = create_prompt(cot_system_message,
                               examples_run, user_message_template)
    # évaluation des deux prompts sur les avis de référence,
    # puis enregistrement des scores micro-F1 dans les deux listes
```

Les trois répétitions donnent un micro-F1 de 1,0 pour les deux prompts, une moyenne de 1,0 et un écart-type de 0,0.

```
Répétition 1/3
Few-shot : 1.0 | CoT : 1.0
Répétition 2/3
Few-shot : 1.0 | CoT : 1.0
Répétition 3/3
Few-shot : 1.0 | CoT : 1.0
```

	Répétition	Few-shot	CoT Few-shot
0	1	1.0	1.0
1	2	1.0	1.0
2	3	1.0	1.0

	Prompt	Moyenne micro-F1	Écart-type
0	Few-shot	1.0	0.0
1	CoT Few-shot	1.0	0.0

FIGURE 11 – Scores des répétitions, moyenne et écart-type pour les prompts Few-shot et CoT

Pour visualiser ces scores, on les représente sur un graphique avec une courbe par prompt. Les deux courbes sont confondues sur la valeur 1,0 pour les trois répétitions.

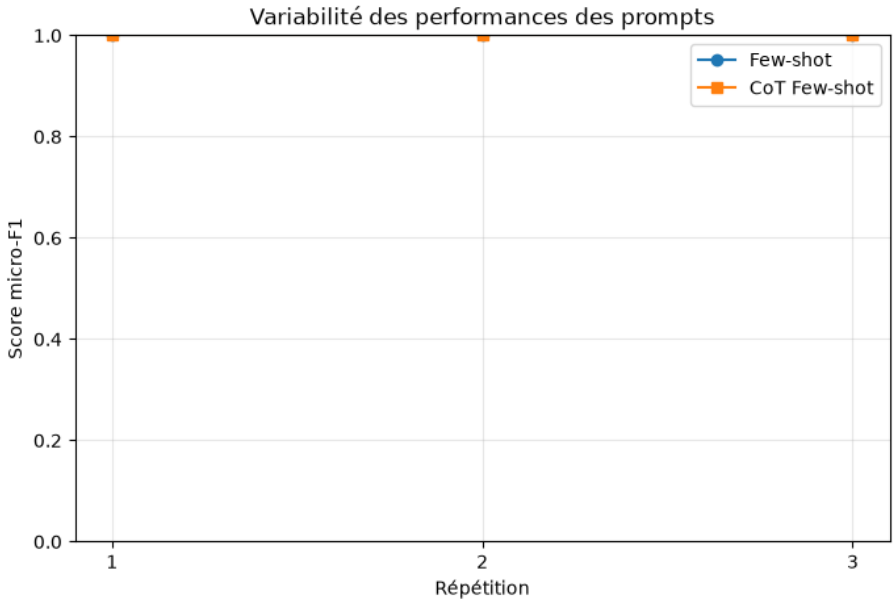


FIGURE 12 – Évolution du score micro-F1 au cours des répétitions

6 Conclusion

Ce TP a permis de prendre en main plusieurs éléments importants pour travailler avec les LLM. On a compris que le texte est découpé en tokens avant d'être traité par le modèle, et que ce découpage ne correspond pas toujours aux mots. On a aussi appris à appeler un modèle de langage depuis un notebook avec LangChain.

La partie sur l'analyse de sentiment a montré comment comparer plusieurs prompts de manière méthodique : préparation d'un ensemble de référence, construction des prompts Zero-shot, Few-shot et CoT, puis évaluation avec le micro-F1 et l'accuracy. Sur les dix avis de référence, les trois prompts ont obtenu un score de 1,0, et les répétitions n'ont pas fait apparaître de variabilité. Ce résultat ne permet pas de les départager : il faudrait un ensemble de référence plus large et davantage de répétitions pour conclure. On retient aussi que la rédaction du prompt (format de sortie imposé, avis délimités, exemples équilibrés et mélangés) permet d'exploiter automatiquement les réponses du modèle, puisqu'aucune réponse n'a été classée comme inconnue.